Supplementary Material for Semantic Atoms for Scalable Text Analysis in Data Analytics in the Era of LLM-Based Agents

Jinxiang Xie
Nanjing University
Nanjing, China
xiejinxiang@smail.nju.edu.cn

Zihao Li
Tsinghua University
Beijing, China
l-zh21@mails.tsinghua.edu.cn

Rui Ding
Microsoft Research
Beijing, China
juding@microsoft.com

Bartuer Zhou
Microsoft Research
Beijing, China
bazhou@microsoft.com

Shi Han
Microsoft Research
Beijing, China
shihan@microsoft.com

Dongmei Zhang
Microsoft Research
Beijing, China
dongmeiz@microsoft.com

1 Additional Discussion and Scope

1.1 What is atomic in TADA?

Tagging and Categorization are atomic at the interface boundary: each exposes one stable output structure to the analytical workflow. Atomicity does not mean that an implementation uses one prompt, one LLM call, or no internal symbolic processing. A Tagging call may perform intent detection, batching, validation, and recovery before returning one string per input item. Categorization may generate local candidates and consolidate them before returning one category list. The caller observes the same contract regardless of these internal choices.

The atom-level query specifies the semantic processing to perform. It may include a perspective, allowed labels, a scale, or a category count, but it does not redefine the output structure. Tagging always returns one string per input item; Categorization always returns category objects with a required name and optional description. This separation lets the semantic task vary without expanding the public API.

1.2 Strings, types, and symbolic validation

Tagging uses strings as a common interchange representation. Depending on the query, a returned string may denote a Boolean decision, category name, number, extracted span, translation, or summary. The symbolic runtime is responsible for converting values to the required type and validating task-specific constraints before using them in filtering, sorting, aggregation, or model fitting. The contract therefore provides representational flexibility but does not by itself guarantee that every returned string is valid for its downstream operation.

1.3 Relationship to semantic operators

Many row-level semantic operators can be decomposed into Tagging followed by an exact operation, and category-discovery workflows can be expressed through Categorization followed, when needed, by Tagging. This decomposition is most useful when later steps need the semantic state itself. A composite operator remains preferable when the final filtered, ranked, joined, or aggregated artifact is the only required result, or when its implementation uses specialized pairwise or retrieval algorithms.

The two atoms are not a completeness claim. Pairwise joins, comparison-based ranking, graph construction, global matching, and other structured outputs may motivate additional atoms. The criterion is a new semantic output structure that cannot be represented through the existing contracts, rather than a new sequence of symbolic operations over outputs the two atoms already expose.

1.4 What scale transparency guarantees

Scale transparency means that the public signature and output structure do not change with input size. It does not assert that a one-call and multi-call execution produce identical semantic values. Partitioning changes the context seen by local calls, candidate retention may discard information, and semantic consolidation may merge or split categories differently. Validation, retries, completion checks, and the large-scale experiments in the main paper characterize these implementation-level risks.

2 Atom and Agent Implementation Materials

2.1 Prompting and model roles

The atom library combines a dedicated system prompt, the atom-level query, and the selected text values. The prompts adapt the Algorithmic Prompting and Thinking-before-Speaking principles used by CAST to the current Tagging and Categorization contracts. The orchestrator model is selected independently: it analyzes the user request, inspects the table schema, writes the complete Python workflow, and responds to execution failures. Atom models execute the narrow, high-volume semantic calls. Recording both model identifiers is therefore necessary to reproduce a TADA-Agent run.

2.2 Programmatic Tool Calling and the text-analysis skill

The text-analysis skill supplies the orchestrator with the two API signatures, input and output conventions, composition rules, and canonical examples. The orchestrator then invokes `ta.tag(data, column, query)` and `ta.categorize(data, column, query)` inside generated Python. Dataframe slices, loop variables, atom outputs, and exceptions remain in the same code-execution environment. The accompanying artifact includes the skill text, atom

Table 1: Public atom contracts and their materialized analytical state.

Atom	Input	Raw return	Materialization	Large-scale path
Tagging	Text sequence X and atom-level query q	One string s_i for each x_i, preserving length and order	One row-aligned column associated with input `rid` values	Independent: partition and reassemble; joint: discover a shared category list, then use the independent path
Categorization	Text sequence X and atom-level query q	Category objects with a required name and optional description	One category table with fresh `cid` values	Partition, retain local candidates, semantically consolidate, and select the requested global list

prompts, planner implementation, and sanitized experiment configurations; raw generated programs from provider-backed runs are not distributed.

2.3 Scale-transparent execution and recovery

For independent Tagging, each chunk is processed under the same query and the library verifies that every input row has exactly one output before reassembling by `rid`. For joint Tagging, large-scale Categorization first establishes a shared category list and the library constructs one assignment query for every chunk. For Categorization, the library shuffles rows, produces local candidate lists under budget r, and semantically consolidates all local lists and pilot support evidence in one global call. The implementation rejects a reducer input that exceeds its conservative context budget rather than introducing a multi-stage reducer.

For each reported large-scale run, we record N, k, n, r, the realized chunk sizes, model identifiers, prompts, random seeds, mapper and reducer calls, retries, completion checks, token usage, and wall time. Missing rows, duplicate identifiers, malformed category objects, under-filled local lists, and failed consolidation calls are execution outcomes rather than ignorable logging events.

3 Top-k Preservation and Bootstrap Planning

3.1 Fixed-taxonomy preservation analysis

Consider the idealized candidate-retention layer used to guide the large-scale Categorization planner. Let $\mathcal{L} = (\ell_1, \dots, \ell_C)$ be fixed canonical categories, and assign each item exactly one category through a partition-invariant function Y. Order the categories by their full-input support:

$$p_i = \frac{1}{N} \sum_{x \in X} \mathbf{1}[Y(x) = i], \qquad p_1 \geq p_2 \geq \cdots \geq p_C. \tag{1}$$

After a random shuffle, let $X_1, \dots, X_B$ be balanced chunks and let $n_{\min} = \min_b |X_b|$. Each chunk computes exact category counts and retains its local top-r:

$$A_b(i) = \sum_{x \in X_b} \mathbf{1}[Y(x) = i], \tag{2}$$

$$H_b = \textsc{TopR}(A_b, r), \tag{3}$$

$$\widehat{T}_k = \textsc{TopK}(\textsc{SumCanonicalCounts}(H_1, \dots, H_B), k)\,. \tag{4}$$

Here TopR returns the r categories with largest chunk counts together with those counts, using a fixed tie rule, and SumCanonicalCounts combines retained counts by immutable category identifier. Assume $k \leq r \leq C$, define $p_{C+1} = 0$, and let

$$\Delta_{i,r} = p_i - p_{r+1}. \tag{5}$$

Let $T_k = \{1, \dots, k\}$ denote the full-input top-k.

Theorem 3.1 (Top-k preservation). *Assume $p_k > p_{k+1}$; Y is fixed independently of the random partition; each chunk computes exact counts and exact local top-r categories; and the reducer sums retained counts by canonical identifier. Then*

$$\Pr\big(\widehat{T}_k = T_k\big) \geq 1 - B(C - r) \sum_{i=1}^{k} \exp\left(-\frac{n_{\min}\Delta_{i,r}^2}{2}\right), \tag{6}$$

where the probability is over the random partition, conditional on $\mathcal{L}$ and Y.

Proof. Let $\hat{S}(i) = \sum_{b=1}^{B} A_b(i)\mathbf{1}[i \in H_b]$ be the count observed by the reducer. Suppose no positive count of a full-input top-k category is discarded. The chunks partition X, so $\hat{S}(i) = N p_i$ for every $i \leq k$, whereas $\hat{S}(j) \leq N p_j$ for every $j > k$. Hence

$$\hat{S}(i) \geq N p_k > N p_{k+1} \geq N p_j \geq \hat{S}(j) \qquad (i \leq k < j),$$

and the reducer returns T_k.

If $A_b(i) > 0$ for some $i \leq k$ but $i \notin H_b$, then at least one retained category $j > r$ ties or overtakes it. For a random chunk, the difference $\mathbf{1}[Y(x) = i] - \mathbf{1}[Y(x) = j]$ lies in $[-1, 1]$ and has mean at least $\Delta_{i,r}$. Hoeffding's inequality for sampling without replacement bounds this inversion by $\exp(-|X_b|\Delta_{i,r}^2/2)$ [1]. A union bound over B chunks, k protected categories, and $C - r$ tail categories yields Equation 6. Ties are included in the failure event. When $r = C$, no category is discarded and recovery is exact.

For $r < C$, the smallest protected gap gives the sufficient condition

$$n_{\min} \geq \frac{2}{\Delta_{k,r}^2} \ln\left(\frac{Bk(C - r)}{\delta}\right) \quad \Longrightarrow \quad \Pr(\widehat{T}_k = T_k) \geq 1 - \delta. \tag{7}$$

The condition uses the realized minimum chunk size $n_{\min} = \lfloor N/B \rfloor$ with $B = \lceil N/n \rceil$.

3.2 Theory-guided operational bootstrap

The sufficient bound explains which quantities govern retention but can be vacuous and is sensitive to plugging noisy pilot frequencies into a squared gap. TADA therefore records the plug-in bound as a diagnostic and uses a paired bootstrap to select r.

Let the pilot produce canonical assignments $S^{\mathrm{p}} = (s_1^{\mathrm{p}}, \dots, s_{N_{\mathrm{p}}}^{\mathrm{p}})$. For each outer bootstrap replicate u, the planner:

(1) draws a support profile from the Jeffreys Dirichlet posterior Dirichlet$(c_1 + \frac{1}{2}, \dots, c_C + \frac{1}{2})$, where c_i is the pilot count of category i;
(2) draws multiple random balanced-partition count tables under the resampled profile;
(3) for each partition simulation v, computes the realized full-input top-k, denoted $T_k^{(u,v)}$;
(4) for every candidate $r \in \mathcal{R}$, retains local top-r counts, combines them by canonical identifier, and records whether $\widehat{T}_k^{(u,v)} = T_k^{(u,v)}$.

All candidate values share the same support profiles and partition draws. If $Z_{u,v}(r) \in \{0, 1\}$ is the recovery indicator, the conditional recovery for outer replicate u is

$$\hat{\rho}_u(r) = \frac{1}{V} \sum_{v=1}^{V} Z_{u,v}(r).$$

The implementation takes a lower empirical quantile $Q_\alpha(r)$ across $\hat{\rho}_1(r), \dots, \hat{\rho}_U(r)$ and selects the smallest candidate whose quantile meets the target recovery. This quantile represents pilot-profile uncertainty; the finite number of partition simulations is recorded separately as Monte Carlo error rather than being presented as a deployment guarantee. This makes the theoretical connection explicit without treating either the conservative sufficient bound or simulation error as a calibrated probability estimate.

The planner records, for every candidate r, the empirical gap $\hat{p}_k - \hat{p}_{r+1}$, the plug-in theorem lower bound, bootstrap mean recovery, lower pilot-profile quantile, Monte Carlo diagnostics, and candidate-retention volume Br. If the pilot exposes no category below rank k, r is not identifiable from the observed tail; the implementation uses a conservative configured budget and records that fallback.

3.3 Scope of the analysis

The theorem and bootstrap isolate candidate retention under one canonical label per row and exact consolidation. Live Categorization additionally contains semantic discovery, local underfilled lists, synonymous category names, and consolidation error. The experiments therefore report an evaluation with known full-population labels, a fixed semantic-assignment layer, and a separate end-to-end open-discovery layer rather than attributing every failure to n or r. The bootstrap also treats the observed pilot category list as the available tail and the pilot Tagging outputs as canonical. Unseen categories, pilot-to-full taxonomy drift, Tagging error, and the difference between count-ranked local top-r and LLM-ranked category proposals are reported as limitations rather than absorbed into the retention estimate.

4 Neural–Symbolic Boundary Diagnostic

We use a controlled diagnostic to test the execution boundary motivating TADA. Each task receives the full synthetic table in one GPT-4.1 call. The *monolithic* condition asks the model to perform semantic processing together with sorting, filtering, or aggregation. The *atom+symbolic* condition uses Tagging to extract the row-level semantic values and Python to perform the exact operation. No agent orchestration, chunking, consolidation, or recovery is used.

Tagging followed by symbolic code is exact for all three tasks at every tested size. The monolithic sort is incomplete at every size, the filter degrades sharply as N grows, and every monolithic aggregate is numerically incorrect. At $N = 1000$, the sort output also contains three duplicate identifiers.

This diagnostic isolates the neural–symbolic boundary on templated inputs with rule-based answers. Each scale point has one GPT-4.1 execution, and the monolithic condition uses a raw prompt rather than a mature semantic-operator system; it therefore illustrates a mechanism rather than estimating comparative system performance. The artifact contains the inputs, prompts, parsed and raw outputs, scripts, manifest, and checksums.

5 Public-Benchmark Protocol Details

5.1 CUAD

CUAD is implemented as row-aligned clause Tagging followed by deterministic normalization into the official evaluation format. The main text reports 0.787 average F1 for TADA-Agent versus 0.753 for DocETL and 0.619 for LOTUS under the common evaluation environment.

5.2 BioDEX

BioDEX contains 100 drug-safety reports evaluated against the 24,313-term MedDRA vocabulary. TADA first extracts free-text adverse-reaction phrases through Tagging and then deterministically aligns them to the top-200 retrieved candidate terms. Three saved TADA runs yield mean (SD) values of 0.2295 (0.0009) RP@5, 0.2465 (0.0095) RP@10, and 0.3849 (0.0104) term recall. Their 95% t intervals are [0.2273, 0.2317], [0.2229, 0.2700], and [0.3591, 0.4107], respectively. The protocol-matched DocETL point is (0.201, 0.196, 0.305) and the LOTUS point is (0.194, 0.200, 0.334), ordered as RP@5, RP@10, and term recall. Other saved LOTUS runs use different prompting and retrieval settings and are excluded from the matched comparison.

6 TABench Construction, Task Suite, and Evaluation

6.1 Taxonomy-derived construction procedure

TABench was constructed from a literature-grounded taxonomy of user intents over free-text columns. The taxonomy has nine top-level classes: classification and labeling, open grouping and clustering, semantic filtering, information extraction and text structuring, aggregation and summarization, joint analysis of text and structured columns, semantic ranking, anomaly detection and data repair, and semantic augmentation. For benchmark annotation, three broad classes are split where their output contracts

Table 2: Single-run neural–symbolic boundary diagnostic. Sort coverage reports returned identifiers over requested rows; aggregation reports exact agreement on count, sum, and average.

N	Monolithic sort	Monolithic filter F1	Monolithic agg.	Atom+symbolic
100	79/100	1.000	No	Exact
200	161/200	0.765	No	Exact
500	280/500	0.678	No	Exact
1000	400/1000	0.105	No	Exact

differ materially: field extraction versus text-to-table structuring, numerical/temporal analysis versus semantic join, and anomaly detection versus repair. This yields the 12 scenario classes used in the main paper (S1–S12 in Table 3): S1 row-wise classification, S2 open grouping and clustering, S3 semantic filtering, S4 field extraction, S5 text-to-table structuring, S6 aggregation and summarization, S7 numerical/temporal analysis, S8 semantic join, S9 semantic ranking, S10 anomaly detection, S11 normalization and repair, and S12 constrained generation and augmentation.

Case construction followed four stages. First, each scenario class was mapped to at least one realistic workbook intent and to the semantic and deterministic operations needed to answer it. Second, source workbooks were generated with stable row keys and with enough structured context to exercise downstream filtering, grouping, joining, aggregation, or ranking rather than only an isolated LLM call. Third, each request received an explicit output contract and query-specific rubric covering schema, row or group coverage, deterministic computation, semantic content, and usability. Fourth, cases were annotated along analytical scenario, semantic operation, and input–output structure.

The suite was built in two rounds. A seed round produced 26 cases, one per distinct taxonomy leaf sub-scenario then covered. An expansion round added 17 cases targeting the remaining independent leaf sub-scenarios, in two groups: new queries over already-frozen source workbooks (each on a distinct workbook, with answers not derivable from the original task's output), and newly generated workbooks whose generator injects controlled phenomena (near-duplicate paraphrase clusters, entity-alias variants, format-diverse numeric and date fields, misleading auxiliary columns) and therefore carries deterministic gold labels alongside the judge protocol. Three seed cases probing text-to-table structuring, semantic imputation, and semantic join were excluded from the evaluated suite because their required output structure falls outside the two-atom contract; they are discussed as boundary evidence in the main paper's Discussion section rather than scored. The resulting evaluated suite contains 40 cases. It is a coverage-driven diagnostic sample, not a claim about the empirical frequency of all real-world spreadsheet requests.

6.2 Task inventory

The frozen TABench release contains 43 source-workbook and natural-language-request pairs. The scored evaluation uses 40 of them, organized by the scenario classes of the taxonomy (Table 3). Ten of the 12 classes are represented; text-to-table structuring (S5) and semantic join (S8) are the boundary classes excluded above; the remaining excluded request probes semantic imputation, another output contract not represented by the current atoms. Every covered class contains at least three cases; several requests additionally stress multiple columns, cross-row context, or misleading auxiliary signals. The suite therefore evaluates complete semantic–symbolic workflows rather than isolated calls to a preselected semantic operator.

6.3 Contract checks and rule-based answers

The normalization-and-repair class (S11) is reported here because its row-level metric evaluates a single case; the remaining cases use rule-based contract checks. On the row-level case all five methods lie between 99.8 and 100.0 with fully overlapping intervals. On the seven cases with rule-based answers, TADA-Agent scores exactly 100% on all of them; the semantic-operator baseline is at 100% on five, with 99.9±0.1 and 98.2±3.2 on the remaining two.

6.4 Semantic-operator execution and request ledger

The semantic-operator baseline runs through the library's official `sem_*` APIs with unchanged prompts, model, temperature, and batch size on the same fixed endpoint. Two engineering accommodations were required and are disclosed: the library's `sem_extract` JSON mode is incompatible with the serving endpoint's structured-output interface, so affected cases use a semantically equivalent per-field `sem_map` fallback; and one long-running case was completed through 25-row checkpointed chunks after a host interruption, with all rows preserved. The request ledger classifies every task-run's evidence as exact (persisted per-call progress), reconstructed (deterministic per-row arithmetic from completed artifacts), or bounded (randomized traces with provable ranges). Judge calls and host-agent turns are excluded. The reported total of at least 18,852 requests is a strict lower bound: one failed task-run defines 400 planned per-row requests but preserves no durable progress and no deliverable, so it contributes zero; failed and recovery attempts without persistent usage are likewise excluded. Full token totals are not reported because per-request usage was not persistently recorded by the library, and wall time is not used as evidence because endpoint and substrate latency confound it.

6.5 Judge protocol and calibration

The unified protocol records per-case judgment-unit counts, disputed-unit fractions, per-run net verdicts, the adjacent-level tie rule for fixed integer rating scales, and the stratified human-calibration design. The controlled matrix comprises 17,949 row-aligned judged units (107,694 judge calls) and 266 case-level units

Table 3: TABench task inventory by scenario class. Dominant program structure describes the typical composition; a query may exercise more than one class's mechanics. S5 and S8 are boundary classes outside the two-atom contract and are discussed rather than scored.

ID	Scenario class	Cases	Dominant program structure
S1	Row-wise classification	Q01–Q03, Q27, Q28	Tag typed labels against a fixed or inferred label set (one string value per row per call).
S2	Open grouping & clustering	Q04, Q05, Q36	Categorize a corpus-level taxonomy, optionally Tag rows with its identifiers; variants add outlier isolation.
S3	Semantic filtering	Q06–Q08, Q29, Q38	Tag a predicate per row, then deterministic row selection; includes negated, implicit, and alias-sensitive predicates.
S4	Field extraction	Q09, Q10, Q30, Q39, Q40	Tag typed fields per row from free text; repeated per output column; includes open-set and relational attributions.
S6	Aggregation & summarization	Q13–Q15, Q31	Group rows deterministically, Categorize or summarize per group, compare across groups or time.
S7	Numerical/temporal analysis	Q16, Q17, Q32, Q42	Combine semantic state with numeric filtering, aggregation, or multi-column fusion.
S9	Semantic ranking	Q19, Q20, Q33, Q34	Tag scores on the requested numerical scale or ordering constructs, then deterministic sort.
S10	Anomaly detection	Q21, Q22, Q37	Categorize or Tag to expose outliers, near-duplicates, or emerging segments.
S11	Normalization & repair	Q23, Q44, Q45	Tag canonical values or suspect flags preserving the original column.
S12	Constrained generation & augmentation	Q24–Q26, Q35	Tag generated columns under explicit output constraints.

(1,596 calls); the semantic-operator comparison adds further row-aligned and case-level units judged under the identical protocol, recorded per case in the released artifacts.

Stratified human calibration. The calibration sample contains 39 judgment units drawn from the row-aligned tasks: 21 from contested units (where per-run judge verdicts disagreed or direction majorities conflicted), 12 from low-confidence units, and 6 from high-confidence units. Six human annotators independently and blindly labeled every unit under the same anonymized A/B presentation as the judge (A better / B better / tie), without access to system identities or judge verdicts. Annotators reach a majority verdict on 33 of 39 units; the judge matches the human majority on 26 of these 33 (78.8%, Cohen's $\kappa = 0.68$), with two direction reversals and the remainder tie-versus-preference disagreements. Table 4 breaks the comparison down by stratum and task. Inter-annotator agreement is itself moderate (mean pairwise raw agreement 0.55, Fleiss $\kappa = 0.32$), and the judge's mean raw agreement with individual annotators (0.60) falls within the annotator pool's own range (0.36–0.77). All six majority-free units fall on fixed-scale rating tasks where the candidates differ by one scale step, the regime absorbed by the deterministic tie-band rule. The sample was drawn from an earlier calibration pool and includes Q12, which is used only for judge calibration and does not enter the final 40-case system scores.

Table 4: Human calibration of the LLM judge on 39 stratified judgment units, six annotators. *Maj.* = units with a human majority verdict; *Agree* = judge matches human majority; *Flip* = directional conflict (one side A, the other B).

	Units	Maj.	Agree	Flip
By stratum				
Contested	21	19	17	1
Low-confidence	12	8	4	1
High-confidence	6	6	5	0
By task type				
Summarization (Q24)	8	8	8	0
Translation (Q25)	7	7	7	0
Document revision (Q12)	6	5	1	1
Relevance rating (Q20)	6	4	2	1
Multi-column extraction (Q10)	4	4	4	0
Priority rating (Q26)	4	1	1	0
Sentiment rating (Q19)	3	3	2	0
Urgency triage (Q02)	1	1	1	0
Total	39	33	26	2

6.6 Error-analysis procedure

After scoring, evaluator notes and available traces were inspected to classify recurring failure modes. These include lexical or rule-based substitution for a semantic judgment, incoherent open-ended categories, incomplete row coverage, incorrect symbolic post-processing, missing workbook mutations, and tool failures. These categories are diagnostic rather than additional scoring rules.

7 Large-Scale End-to-End Execution Details

This appendix documents per-repetition execution and cost telemetry for the RQ2 end-to-end comparison in the main paper. All rows correspond to saved artifacts; nothing is estimated except where explicitly marked.

7.1 Per-repetition cost telemetry

Table 5 lists every repetition. The three systems are metered differently, and the table preserves those distinctions rather than merging them: TADA-Agent and LOTUS token counts are provider-reported physical usage; Claude Code counts are recovered from its session transcripts (one entry per assistant API message), where token totals are dominated by prompt-cache reads and three repetitions have incomplete token fields (marked ∗), which we leave unavailable rather than estimate.

Table 5: Per-repetition execution cost. Calls: planned LLM calls for TADA-Agent (observed attempts in parentheses where they differ), API messages for Claude Code, per-row calls for LOTUS. Tokens in thousands; "cache" is Claude Code prompt-cache reads. Wall time in minutes. ∗: transcript token fields incomplete; –: not recorded or not aggregable.

Workload	Rep	Calls	Prompt	Compl.	Cache	Wall
TADA-Agent (provider-reported)						
Sentiment	1	33 (105)	–	–	–	164.6
Sentiment	2	33	891.7	349.2	–	15.1
Sentiment	3	33	891.7	259.9	–	12.2
Topic	1	33	894.7	623.5	–	29.2
Topic	2	33	894.7	674.5	–	31.8
Topic	3	33	894.7	656.0	–	36.8
Category	1	7	242.5	49.5	–	2.3
Category	2	7	242.5	56.5	–	9.6
Category	3	7	242.5	95.4	–	5.4
Claude Code (transcript-recovered)						
Sentiment	1	67	93.1	51.0	39.4	9.1
Sentiment	2	99	131.6	65.1	734.6	10.0
Sentiment	3	40	5.5*	20.7	12.0*	29.4
Topic	1	40	57.9	35.0	20.6	7.9
Topic	2	106	71.2	27.6	441.4	106.2
Topic	3	155	260.2	49.9	850.5	67.9
Category	1	22	16.9	12.3	21.3	2.0
Category	2	57	39.4	23.2	142.8	3.9
Category	3	29	–*	15.3	–*	2.4
LOTUS `sem_map` *(provider-reported)*						
Sentiment	s500	500	59.8	2.5	–	39.3
Topic	s500	500	91.8	2.8	–	33.7
Category	p2050	2,050	–	–	–	–

Three notes qualify the table. First, TADA-Agent sentiment repetition 1 is the telemetry exception disclosed in the main text: transient provider failures inflated the planned 33 calls to 105 observed request events, the wall time to 164.6 minutes, and the provider token report was lost. The chunking plan was unchanged; repetitions 2–3 executed it exactly. Second, the Claude Code topic repetition shown as repetition 2 replaced an earlier attempt that exceeded a three-hour timeout without producing output. Third, the LOTUS category row aggregates the first 2,050 frame rows of a checkpointed full-frame run that spanned several resumed processes at varying concurrency; its calls are exact but wall time and tokens are not aggregable across processes and are therefore omitted.

7.2 Full-scale cost estimates for the per-row baseline

LOTUS `sem_map` issues one LLM call per input row, so per-row cost is additive and the measured subsets scale linearly. From the 500-row measurements (119.6/183.6 prompt and 5.0/5.5 completion tokens per row; 4.7/4.0 seconds per row at concurrency one), one full 32,179-row repetition is estimated at 32,179 calls (exact by definition; 975× TADA-Agent's 33), 4.0M tokens for sentiment and 6.1M for topic (3–5× TADA-Agent's measured 1.2–1.6M), and roughly 36–42 hours of wall time at the same serving concurrency (>100×). Call count is exact; token and time figures are linear extrapolations of measured per-row cost, not measurements.

7.3 Coding-agent repetition artifacts

Each controlled coding-agent repetition leaves a saved output file, an exit status, and a session transcript. Transcript-recovered telemetry (Table 5) shows the agent's self-chosen execution structure varies widely across completed repetitions of the same task: 22–155 assistant API messages, with heavier repetitions corresponding to iterative script debugging and re-reads of intermediate files (visible as cache-read-dominated token totals). One topic repetition exceeded a three-hour budget without producing output and was replaced by a completed rerun under the identical protocol; the replacement is the repetition reported in the main table.

7.4 Open-vocabulary mapping protocol

In the open-vocabulary variant, the coding agent must first discover its own category vocabulary. Because discovered labels need not match reference names, we score them through an explicit semantic label mapping: each discovered label is mapped to the closest reference category, or to none when no reference category corresponds; rows whose labels map to none are counted as errors against the full denominator. Table 6 lists every repetition. Unmapped labels are of two kinds: granularity merges (one discovered label spanning several reference categories, e.g. "bugs, crashes & performance") and invented catch-all buckets ("general criticism & complaints"). The saved mapping tables and unmapped-label lists accompany the artifacts.

Table 6: Open-vocabulary repetitions: discovered vocabulary size, labels with no reference correspondence, and mapped accuracy.

Workload / rep	Discovered	Unmapped	Mapped acc.
Topic 32K / 1	10	4	0.690
Topic 32K / 2	10	3	0.675
Topic 32K / 3	10	7	0.685
Category 6.6K / 1	12	8	0.296
Category 6.6K / 2	11	6	0.134
Category 6.6K / 3	10	4	0.195

8 Adaptive Retention Evaluation Details

8.1 Controlled distributions

The controlled retention experiment isolates the fixed-taxonomy retention stage analyzed in Appendix 3. It uses $N = 32{,}000$ fixed

canonical assignments over $C = 20$ categories, requests the global top-5, and partitions the rows into $B = 32$ chunks of $n = 1{,}000$. No LLM call occurs during the retention replay. This is deliberately separate from open Categorization, where semantic discovery and consolidation introduce additional errors.

The six populations are generated from explicit descending rank weights. In every case,

$$(w_1, \dots, w_5) = (2.00, 1.75, 1.50, 1.25, 1.00), \qquad p_i = \frac{w_i}{\sum_j w_j}.$$

For the separated cutoff,

$$w_6 = 0.40, \qquad w_i = 0.30(0.85)^{i-7}, \quad i = 7, \dots, 20.$$

The five narrow levels share

$$w_i = 0.75(0.88)^{i-9}, \quad i = 9, \dots, 20,$$

and use the following near-cutoff weights:

	w_6	w_7	w_8
1	0.965	0.955	0.945
2	0.970	0.960	0.950
3	0.975	0.965	0.955
4	0.980	0.970	0.960
5	0.982	0.972	0.962

The separated gap is $p_5 - p_6 = 0.061891$; the five narrow gaps are 0.002293, 0.001963, 0.001634, 0.001306, and 0.001175. We allocate $\lfloor N p_i \rfloor$ rows to each category and distribute the remaining rows by largest fractional remainder, producing one fixed 32,000-row population per case. The released six CSV files contain every row ID and canonical category.

For each population, a 1,000-row sample supplies the planner with canonical assignments. The planner draws 100 outer support profiles from a Jeffreys Dirichlet posterior and evaluates 20 paired random partitions per profile for every candidate $r \in \{5, \dots, 10\}$. It chooses the minimum certified candidate when one exists; otherwise it records a conservative fallback to the configured maximum $r = 10$.

8.2 Connection to the preservation analysis

The theorem and planner use the same quantities: $n_{\min}$, B, k, r, and the gap from a protected top-k category to rank $r + 1$. The plug-in sufficient bound is zero at both selected operating points, so it is not a calibrated certificate. It still predicts the qualitative response: larger cutoff gaps and chunks reduce local rank inversions, while retaining past a near-tied plateau increases the relevant gap.

The paired bootstrap supplies the operational estimate. Outer profiles represent pilot-distribution uncertainty; inner simulations estimate random-partition recovery. Shared draws across r reduce comparison noise. The separated case needs no extra budget. Across the five narrow levels, the minimum reliable r is 9 or 10 and the planner recommends 10: exact in the two hardest levels, conservative by one category in the other three, and never too small. Recommended-point recovery ranges from 98.5% to 100%.

Why not always retain all categories? If recovery were the only objective, setting $r = C = 20$ would remove candidate-retention loss by construction. The planner instead solves a constrained cost problem: find the smallest local budget that reaches the target reliability. Here $B = 32$, so $r = 10$ sends 320 category objects to the reducer, whereas $r = 20$ sends 640. The latter also increases mapper output tokens, reducer context, latency, and—in open Categorization—the number of low-support or synonymous proposals that semantic consolidation must resolve. For the hardest narrow level, increasing r from 10 to 20 raises empirical recovery from 98.5% to 100% while doubling candidate volume. Thus the selected r is the minimum reliable budget, not the unconstrained quality-maximizing value.

Shared curve shape and the dip at $r = 8$. The five orange curves look similar by design: they form a controlled family that changes only the cutoff gap while keeping the three-category near-cutoff structure and geometric tail fixed. Empirical recovery need not be monotone in r because the reducer sums only retained local counts. Changing r changes which categories have nearly complete versus systematically truncated counts. In narrow level 5, at $r = 8$ the average retained-count fractions for global ranks 5–8 are approximately 96.8%, 95.6%, 94.7%, and 93.9%. The three competitors are therefore almost fully represented while the true rank-5 category still loses enough count for its small global margin to be reversed. At $r = 9$–10, all four fractions exceed 99% and recovery rises sharply. We therefore define the minimum reliable value as the first r whose recovery and every larger tested r remain above the target, rather than the first isolated crossing.

Role of the analysis. At these operating points the sufficient bound is vacuous, so the bootstrap is the selection mechanism. The bound nevertheless identifies the relevant rank-sensitive margin $\Delta_{k,r} = p_k - p_{r+1}$ rather than only the adjacent gap $p_k - p_{k+1}$, explaining why retaining past a near-tied plateau can help. It also exposes the scaling variable $n_{\min}\Delta_{k,r}^2$ and the dependence on B, k, and tail size, which determine the bootstrap simulation and the controlled distributions. It supplies a distribution-free sufficient certificate in regimes where the bound is nonvacuous and proves exact recovery at $r = C$, while separating candidate-retention error from semantic assignment and consolidation error.

8.3 Scope and evidence

The controlled experiment characterizes candidate retention under fixed canonical assignments, complementing the fixed-taxonomy end-to-end Tagging evaluation in the main paper. Its protocol, population-generating distributions, and aggregate recovery results are reported here. The later trial-level bundle was not present in the frozen artifact handoff and is not represented as part of the public repository.

9 Reproducibility Materials

The repository README and per-directory scope notes distinguish executable code, recovered run artifacts, aggregate evidence, and intentionally absent raw provider traces. A SHA-256 manifest covers the released experiment artifacts.

References

[1] Wassily Hoeffding. 1963. Probability Inequalities for Sums of Bounded Random Variables. *J. Amer. Statist. Assoc.* 58, 301 (1963), 13–30. https://doi.org/10.1080/01621459.1963.10500830

Table 7: Planner position relative to the full $r = 5, \ldots, 20$ recovery matrix. The oracle is the first r whose recovery and every larger tested r remain at least 0.95. Empirical recovery uses 5,000 partitions of each fixed population.

Regime	$p_5 - p_6$	Oracle $r^\star$	$\hat{r}$	$\widehat{Q}_{05}$	Rec. at $r = 5$	Rec. at $\hat{r}$
Separated	0.061891	5	5	1.00	1.000	1.000
Narrow 1	0.002293	9	10	0.90	0.909	1.000
Narrow 2	0.001963	9	10	0.95	0.871	1.000
Narrow 3	0.001634	9	10	0.95	0.824	1.000
Narrow 4	0.001306	10	10	0.95	0.744	0.990
Narrow 5	0.001175	10	10	0.95	0.739	0.985

Table 8: Reproducibility materials and public-release scope.

Component	**Public materials and scope**
TABench controlled run	Benchmark inputs and rule-based answers; checksums; system configuration; skill and prompts; evaluation and aggregation code; aggregate judgment tables; human-calibration units and labels. Raw provider traces are omitted.
Semantic-operator runs	API paths, evaluation scripts, aggregate outputs, and request-accounting rules. Private endpoints and raw provider telemetry are omitted.
CUAD/BioDEX	Recovered experiment scripts and BioDEX manifests, predictions, and summaries. The CUAD handoff contains implementation and smoke evidence but not the final three full prediction files.
Large-scale execution	End-to-end run records, voted labels and taxonomies, aggregate metrics, and scoring scripts. The later candidate-retention trial bundle is not distributed.
Boundary/theory diagnostic	Protocol and aggregate outcomes in this supplement. Trial-level files were not present in the frozen handoff and are not reconstructed.